Ex. No: 6

Title: Gradient Descent, SGD, Momentum and Adam Comparison

Packages: NumPy, Matplotlib

AIM:

To implement Gradient Descent, SGD, Momentum and Adam optimizers for linear regression and compare their performance.

ALGORITHM:

1. Start the program and import NumPy and Matplotlib.
2. Create data using y = 3x + 2 with small noise.
3. Define the gradient of the Mean Squared Error loss.
4. Gradient Descent: update w and b using the gradient of the full data.
5. SGD: update w and b using one random sample at a time.
6. Momentum: update using a velocity term that stores past gradients.
7. Adam: update using the average of the gradient and the squared gradient.
8. Train all four optimizers for 300 epochs and print the final w, b and loss.
9. Plot the loss of all optimizers and stop.

PROGRAM:

```
import numpy as np
import matplotlib.pyplot as plt

np.random.seed(0)
X = np.random.rand(100)
y = 3 * X + 2 + np.random.randn(100) * 0.1      # true w=3, b=2

def grad(w, b, Xs, ys):
    e = w * Xs + b - ys
    return np.array([np.mean(2 * e * Xs), np.mean(2 * e)])

def train(method, lr, epochs=300):
    w, b, t = 0.0, 0.0, 0
    v, m, s = np.zeros(2), np.zeros(2), np.zeros(2)
    losses = []
    for _ in range(epochs):
        if method == "SGD":
            batches = [[i] for i in np.random.permutation(100)]   # one sample at a time
        else:
            batches = [np.arange(100)]                            # full data
        for idx in batches:
            g = grad(w, b, X[idx], y[idx])
            t += 1
            if method == "Momentum":
                v = 0.9 * v - lr * g
                step = v
            elif method == "Adam":
                m = 0.9 * m + 0.1 * g
                s = 0.999 * s + 0.001 * g ** 2
                step = -lr * (m / (1 - 0.9 ** t)) / (np.sqrt(s / (1 - 0.999 ** t)) + 1e-8)
            else:
```

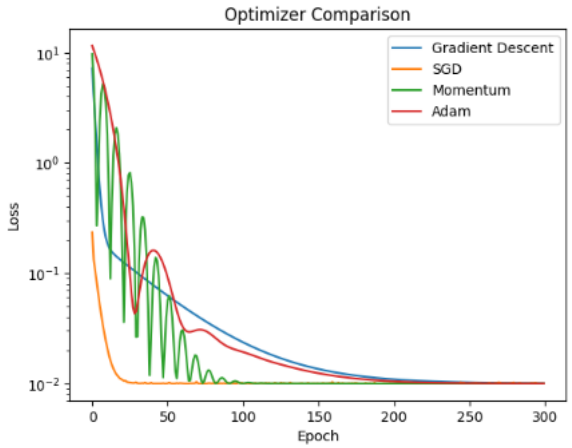
```
            step = -lr * g
        w, b = w + step[0], b + step[1]
    losses.append(np.mean((w * X + b - y) ** 2))
  return w, b, losses

settings = {"Gradient Descent": 0.1, "SGD": 0.01, "Momentum": 0.05, "Adam": 0.1}
for name, lr in settings.items():
  method = "GD" if name == "Gradient Descent" else name
  w, b, losses = train(method, lr)
  print(name, ": w =", round(w, 3), ", b =", round(b, 3), ", loss =", round(losses[-1], 4))
  plt.plot(losses, label=name)

plt.yscale("log")
plt.xlabel("Epoch")
plt.ylabel("Loss")
plt.legend()
plt.title("Optimizer Comparison")
plt.show()
```

OUTPUT:

```
Gradient Descent : w = 2.966 , b = 2.036 , loss = 0.01
SGD : w = 2.997 , b = 2.025 , loss = 0.0099
Momentum : w = 2.994 , b = 2.022 , loss = 0.0099
Adam : w = 2.985 , b = 2.027 , loss = 0.0099
```



RESULT:

All four optimizers were implemented and compared. Each one learned values close to the true w = 3 and b = 2, and the loss graph shows how fast each one converges.